

FACETpy CLI Overview

FACETpy CLI Overview

The FACETpy CLI is built around several subcommands in cli.py:

1. `process` — run EEG correction pipelines
2. `to-bids` — convert corrected outputs into BIDS
3. `viewer` / `view` — visualize raw EEG recordings
4. `analysis` — run data checks and metrics
5. `modes` — list correction/add-on modes
6. `patterns` — list pipeline patterns

```
facetpy-run --help
facetpy-run modes
facetpy-run patterns
```

process command

This is the main entrypoint for artifact correction.

High-level workflow

The command builds a pipeline from:

1. a top-level `--pattern`
2. a template `--correction-mode`
3. optional add-on `--mode` values
4. additional pipeline options

Input sources

You can pass inputs in three ways:

1. `--input file1 --input file2 ...`
2. `--input-list list.txt`
3. `--input-dir path/to/dir`

Batch inputs are resolved, deduplicated, and processed in order.

Recursive scanning

Use `--recursive` with `--input-dir` to search subfolders:

```
--input-dir data/ --recursive
```

The CLI scans files and MFF folders under the directory tree.

Supported extensions

`--extensions` customizes which file types are included when scanning directories.

Patterns

Patterns define the overall pipeline shape.

(1) `--pattern quickstart` (default, don't need to put)

A memory-light workflow:

1. TriggerDetector
2. UpSample
3. pre-template add-ons
4. template correction
5. post-template add-ons
6. DownSample
7. post-downsample add-ons

Use when you want a fast, compact correction pipeline.

(2) `--pattern standard`

The docs standard workflow:

1. TriggerDetector
2. CutAcquisitionWindow
3. HighPassFilter
4. UpSample
5. SliceAligner
6. SubsampleAligner
7. pre-template add-ons
8. template correction
9. post-template add-ons
10. DownSample
11. PasteAcquisitionWindow
12. LowPassFilter
13. post-downsample add-ons

Use for the full recommended correction flow.

`--pattern standard` implicitly enables PCA and ANC even if the modes are not passed.

(3) `--pattern bcg`

A Ballistocardiogram-specific pattern:

1. QRSTriggerDetector
2. AASCORrection

Use for QRS-triggered BCG correction rather than scanner-trigger correction.

Correction modes

Choose the core template-subtraction strategy with `--correction-mode`.

Available modes:

(1) `--correction-mode aas`

baseline averaged artifact subtraction, uses:

`--aas-correlation-threshold`,
`--search-window-factor`,
`--realign-after-averaging`,
`--apply-epoch-alpha-scaling`

(2) `--correction-mode farm`

FACET FARM-style template weighting, uses:

`--farm-correlation-threshold`,
`--farm-search-half-window`,
`--farm-search-half-window-factor`

works with `--interpolate-volume-gaps`

(3) `--correction-mode volume-trigger`

volume-trigger template weighting

(4) `--correction-mode slice-trigger`

slice-trigger odd/even template weighting

(5) `--correction-mode corresponding-slice`

average corresponding slice positions across volumes may use `--slices-per-volume` when inference is not possible

(6) `--correction-mode moosmann`

motion-informed template weighting using an SPM realignment parameter file and requires

--motion-rp-file, uses:

--motion-threshold,

--motion-window-size

Add-on modes

Use `--mode` repeatedly to layer additional correction stages. Order is preserved and duplicates are filtered.

Supported add-on modes:

(1) `volume-artifact`

inserts volume-artifact correction before the template subtraction step

(2) `pca`

applies PCA residual cleanup after template subtraction configured with:

`--pca-components,`
`--pca-hp-freq`

(3) `anc`

applies adaptive noise cancellation after downsampling configured with:

`--anc-filter-order,`
`--anc-hp-freq,`
`--anc-mu-factor,`
`--anc-max-gain`

can use the FastRANC C extension via `--anc-c-extension`

How to stack options correctly

Basic process command

```
facetpy-run process --input raw.edf --output-dir corrected --pattern quickstart --on-error continue --overwrite
```

Switch correction mode

```
facetpy-run process --input raw.edf --output-dir corrected --pattern standard --input raw.edf --on-error continue --overwrite
```

Add PCA and ANC

```
facetpy-run process \  
  --input raw.edf \  
  --output-dir corrected \  
  --pattern standard \  
  --correction-mode farm \  
  --mode pca \  
  --mode anc \  
  --on-error continue \  
  --overwrite
```

Add volume-artifact before template correction

```
facetpy-run process \  
  --input raw.edf \  
  --output-dir corrected \  
  --pattern standard \  
  --correction-mode farm \  
  --mode volume-artifact \  
  --mode pca \  
  --on-error continue \  
  --overwrite
```

Use Moosmann motion correction

```
facetpy-run process \  
  --input raw.edf \  
  --output-dir corrected \  
  --pattern standard \  
  --correction-mode moosmann \  
  --motion-rp-file motion.txt \  
  --on-error continue \  
  --overwrite
```

Output control

`--output-dir / --output`

`--output-extension` sets corrected file format (.edf default)

`--flat-output` writes all batch outputs directly into output-dir

otherwise, each source input gets its own subfolder

Memory / chunking options

process uses chunked execution and can choose between:

1. trigger-section chunks (default)
2. fixed-length chunks via `--fixed-length-chunks`

Chunk control:

`--min-chunks`
`--max-chunks`
`--memory-budget-mb`
`--memory-fraction`

Trigger-section control:

`--trigger-section-padding-seconds`
`--trigger-section-min-triggers`
`--trigger-section-gap-seconds`
`--trigger-section-max-sections`

Channel processing:

`--channel-sequential` controls whether high-memory correction steps run one channel at a time

Error handling in batch mode

Know that error will be thrown if no triggers are present in an eeg

`--on-error raise` stops on first failure

`--on-error continue` processes remaining files and records failures

When failures occur, CLI writes:

`processing_error.json` per failed input

`processing_failures.json` at the batch root

Visualization with viewer

Use `viewer` or `view` to inspect raw EEG before or after processing:

```
facetpy-run viewer --viewer-mode mne --input raw.edf --show
```

Options:

- `--output` save a plot image
- `--viewer-mode` `matplotlib|mne`
- `--channel` select one channel
- `--start` start time in seconds
- `--duration` duration in seconds
- `--n-channels` for MNE viewer
- `--scalings` for MNE viewer scale mode
- `--title` custom title

Data quality and metrics with analysis

Run reports and optional metrics:

```
facetpy-run analysis --input raw.edf --metrics
```

Available behavior:

- detect-events optionally detects triggers before checking
- require-triggers treat missing triggers as failure
- strict raise an error when check report fails
- output-j son write structured metadata

Metrics include:

1. SNRCalculator
2. LegacySNRCalculator
3. RMSCalculator
4. RMSResidualCalculator
5. MedianArtifactCalculator
6. FFTAllenCalculator
7. FFTNiazyCalculator
8. MetricsReport

Convert outputs to BIDS with to-bids

Use to-bids to turn corrected outputs into a BIDS dataset.

```
facetpy-run to-bids --input corrected.edf --output-dir bids_root  
--task facetcorrected
```

Key points:

accepts --input, --input-list, --input-dir

supports recursive scanning with --recursive

uses --subject, --session, --task

can optionally --detect-events with --trigger-regex

--overwrite allows replacing existing BIDS files

--on-error continue keeps processing after failures

The CLI derives BIDS subject/run values automatically from input names and can de-duplicate runs as needed.

Recommended stacking patterns

(1) Pick a pattern:

1. quickstart for speed
2. standard for full pipeline
3. bcg for QRS-based BCG correction

(2) Choose a correction mode:

```
--correction-mode aas
--correction-mode farm
--correction-mode volume-trigger
--correction-mode slice-trigger
--correction-mode corresponding-slice
--correction-mode moosmann
```

(3) Add optional stages:

```
--mode volume-artifact before subtraction
--mode pca after template subtraction
--mode anc after downsampling
```

(4) Tune support options:

```
--upsample-factor
--highpass-freq / --lowpass-freq
--search-window-factor
--pca-components
--anc-hp-freq
```

(5) Batch and recursion:

```
--input-dir ... --recursive
control file selection with --extensions
use --flat-output when you want all outputs in one folder
```

Common commands

Carolina - Recursively quickstart all raw data:

```
facetpy-run process \  
  --input /home/cfishmei/projects/facetpy_data/rawdata \  
  --recursive \  
  --extensions .mff \  
  --output-dir output/corrected \  
  --on-error continue \  
  --overwrite
```